

Customer Journey Map

Date	04 July 2026
Team ID	xxxxxxx
Project Name	Credit Card Approval Prediction
Maximum Marks	2 Marks

Customer Journey Map

Map out the customer's experience stage-by-stage, capturing their actions, touchpoints, thoughts, and feelings, along with process ownership and improvement opportunities at each stage.

Phase of Journey	Stage 1 Data Input & Form Validation	Stage 2 ML Pipeline & Model Processing	Stage 3 Decision & Feedback
Actions <i>What does the customer do?</i>	Fills out the 16 personal, employment, and financial details requested on the web input page. Submits the application data while the frontend triggers automated validation filters (e.g., age bounds and asset check ranges).	Waits on the submission screen while the application backend bundles the inputs into a dynamic DataFrame row. Remains passive as the internal processing engine executes matrix mapping and calls the serialized classifier file.	Views their instantaneous credit approval or rejection output screen on the user terminal. Reviews the customized rule-based explanation notes and looks over their rolling session application history list.
Touchpoint <i>What part of the service do they interact with?</i>	The interactive web input interface generated by the Flask application route <code>@app.route('/predict', methods=['GET', 'POST'])</code> . • Form submission error alerts and real-time input status warning flags.	The live backend application environment loading window that handles the incoming data array. Serialized model asset files (<code>model.pkl</code> , <code>scaler.pkl</code> , <code>encoder.pkl</code>) loaded directly inside the server memory.	The visual result presentation interface mapped to the Flask route <code>@app.route('/result')</code> . The sliding chronological query history log tracked at <code>@app.route('/history')</code> (limited to the last 10 entries).

Customer Thought <i>What is the customer thinking?</i>	"Will my steady income profile offset the small historic dip in my credit score history?" "Is this system going to reject my submission if I accidentally enter a negative balance or an invalid age?"	"How is this predictive model combining my text inputs together with my numerical income statements?" "Is the algorithm analyzing my financial background using a simple tree or a highly advanced ensemble classifier?"	"Why did the system compute this specific confidence score, and what logic dictates my personal rejection feedback text?" "Will closing my active browser window permanently clear my credit history profile from this system?"
Customer Feeling <i>What is the customer feeling?</i>	Apprehensive about submitting private, highly sensitive banking documents online but optimistic about getting access to credit lines.	Impatient and anxious due to the opaque, invisible nature of the machine learning model processing their private records.	Relieved to obtain an automated decision immediately, combined with clarity due to the explicit reasoning text provided.

Process Ownership <i>Who is in the lead on this?</i>	Frontend UI/UX Team & Flask Developer: Responsible for the interface layout and server-side data extraction mechanics.	Machine Learning Engineer & Data Scientist: Responsible for the encoders, standard scalers, and trained XGBoost classification models.	Business & Compliance Analyst: Responsible for checking the descriptive logic rules and validating user history storage metrics.
Opportunities <i>How can we improve this stage?</i>	Integrate client-side JavaScript constraints to instantly block invalid values (like Age < 18) before making server requests. • • Add tooltips explaining category entries (like 'Commercial associate') to clear up confusion during user entry.	Optimize training loops using Recall or ROC-AUC targets instead of simple accuracy to better detect high-risk bad debtors. Implement a dynamic visual processing loader to reassure the customer that their data is actively being processed by the system.	Replace the conditional rule blocks with interactive SHAP charts to show the exact mathematical feature impacts. Connect a persistent database engine to save tracking records securely across multiple user sessions long-term.